EN3150 Assignment 03: Resource-Constrained CNN for Edge Image Classification (GTSRB)

Group name: Overfitting | **Date:** 6 October 2026

Index no.	Name	GitHub profile
230016K	ABISHEK L.	github.com/AbishekLesley
230380T	LUCKSHAN S.M.	github.com/Lakshan-TECH
230572J	SAMPAVI G.	github.com/sampavany
230581K	SANTHOSH S.	github.com/Santhosh-04-S

Shared group repository: github.com/Lakshan-TECH/EN3150-A03-Overfitting

Summary

We built an edge-oriented traffic-sign classifier on GTSRB (43 classes, 39,270 images down-scaled to 64×64). Model A is a standard CNN (365,499 trainable parameters, 16.19 M MACs). Model B uses depthwise-separable convolutions and global average pooling (66,827 parameters, 4.16 M MACs: 5.5× and 3.9× fewer). After 20 epochs Model A reached 99.81% test accuracy and Model B 98.64%. Fine-tuned MobileNetV2 (2.28 M parameters) reached 99.78% and SqueezeNet 1.1 (0.74 M) 99.46%. Model B is 34× smaller than MobileNetV2 for a loss of 1.14 points. On our CPU it was not faster than Model A (1.99 ms against 1.94 ms per image), so its MAC saving did not become a speed saving there.

1. Data Preparation

Dataset. German Traffic Sign Recognition Benchmark (GTSRB): colour photographs of 43 classes of German road signs, taken from a moving vehicle under varied lighting, blur and viewing angle. Source: torchvision.datasets.GTSRB (original host: Institut für Neuroinformatik, Ruhr-Universität Bochum). It is not CIFAR-10. Traffic-sign recognition is a realistic embedded use case (driver-assistance cameras, low-power vision nodes).

Pre-processing.

- The official training and test sets were merged (39,270 images: 26,640 from the official training set and 12,630 from the official test set) and then re-split ourselves so the required ratios are respected.
- Every image was resized to **64×64×3** (bilinear, with PIL anti-aliasing), meeting the maximum-resolution requirement. Original images range from roughly 15×15 to 250×250 pixels.
- **Split:** stratified random split with seed 42: **70% train (27,489 images), 15% validation (5,890), 15% test (5,891)**. Stratification keeps the class proportions identical in all three sets.
- **Normalisation:** per-channel mean and standard deviation computed on the training set only (no information from validation or test): mean (0.340, 0.312, 0.321), std (0.275, 0.264, 0.271).
- **Augmentation (training only):** random rotation ±10°, translation ±10%, zoom ±10%. Horizontal flips were deliberately not used because they change the meaning of directional signs (turn left versus turn right).

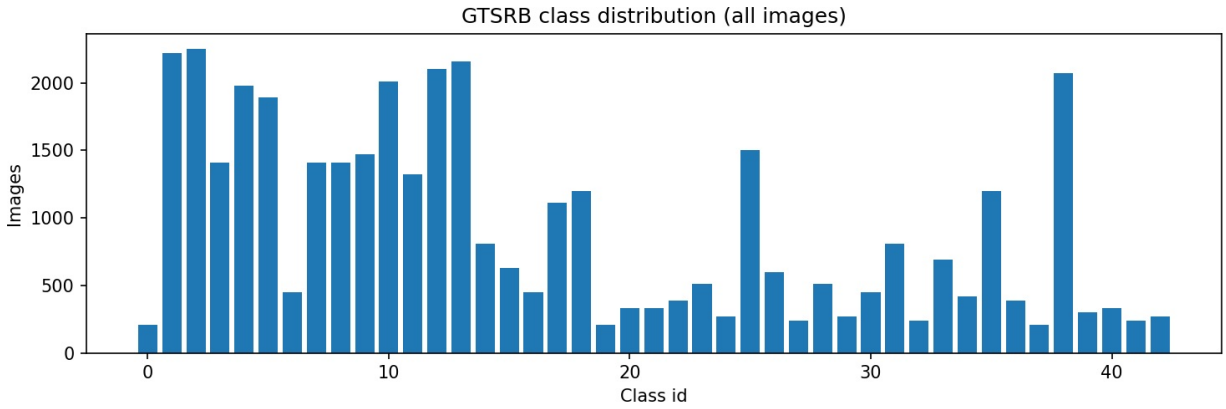


Figure 1: Number of images per class (all 39,270 images). Classes 1 and 2 (speed limits 30 and 50) have about 2,200 images each, while the rarest classes (for example 0, 19 and 37) have about 210, a ratio of roughly 10 to 1.



Figure 2: One sample per class after resizing to 64×64. Many images are dark, low-contrast or blurred (for example classes 6, 17 and 32), which makes the task harder at this resolution.

Dataset sheet: The dataset (GTSRB) was entered in the course dataset sheet under group name Overfitting.

2. Custom Architecture Design

2.1 Parameter-counting rules used

Layer	Trainable parameters
Standard convolution, kernel k, Cin to Cout, no bias	$k^2 \cdot Cin \cdot Cout$
Depthwise convolution (groups = Cin)	$k^2 \cdot Cin$
Pointwise convolution (1×1)	$Cin \cdot Cout$
Batch normalisation (γ and β)	$2 \cdot C$
Fully connected	$in \cdot out + out$

Convolutions are followed by BatchNorm, so their bias is omitted (BN's β replaces it). BatchNorm running mean and variance are buffers, not trainable parameters. MACs count multiplications in convolution and FC layers only (pooling, BN and activations excluded).

2.2 Model A: standard CNN

Four blocks of [Conv 3×3, stride 1, padding 1 → BatchNorm → ReLU → MaxPool 2×2], followed by two fully connected layers.

Layer	Output shape	Weights	BN	Total params	MACs
Input	3×64×64	-	-	0	-
Conv1 3×3, 3→16 + pool	16×32×32	432	32	464	1,769,472
Conv2 3×3, 16→32 + pool	32×16×16	4,608	64	4,672	4,718,592
Conv3 3×3, 32→64 + pool	64×8×8	18,432	128	18,560	4,718,592
Conv4 3×3, 64→128 + pool	128×4×4	73,728	256	73,984	4,718,592
Flatten	2048	-	-	0	-
FC1 (128 units) + ReLU + Dropout 0.3	128	262,272	-	262,272	262,144
FC2 (43 units)	43	5,547	-	5,547	5,504
Total				**365,499**	**16,192,896 (16.19 M)**

Example calculation (Conv3): $3 \cdot 3 \cdot 32 \cdot 64 = 18{,}432$ weights, plus BN $2 \cdot 64 = 128$, giving 18,560. FC1: $128 \cdot 4 \cdot 4 = 2048$ inputs, so $2048 \cdot 128 + 128 = 262{,}272$.

Observation: FC1 alone holds 72% of Model A's parameters. The flatten-then-FC head is the main memory cost, not the convolutions.

2.3 Model B: lightweight CNN with depthwise-separable convolutions

Stem (standard conv) followed by four depthwise-separable blocks. Each block is [Depthwise 3×3 → BN → ReLU6 → Pointwise 1×1 → BN → ReLU6]. Max-pooling after the stem and after the first three blocks halves the resolution. Global average pooling replaces flatten.

Layer	Output shape	Weights	BN	Total params	MACs
Stem Conv 3×3, 3→16 + pool	16×32×32	432	32	464	1,769,472
DS1 depthwise 3×3 (16)	16×32×32	144	32	176	147,456
DS1 pointwise 16→32, then pool	32×16×16	512	64	576	524,288
DS2 depthwise 3×3 (32)	32×16×16	288	64	352	73,728
DS2 pointwise 32→64, then pool	64×8×8	2,048	128	2,176	524,288
DS3 depthwise 3×3 (64)	64×8×8	576	128	704	36,864
DS3 pointwise 64→128, then pool	128×4×4	8,192	256	8,448	524,288
DS4 depthwise 3×3 (128)	128×4×4	1,152	256	1,408	18,432
DS4 pointwise 128→256	256×4×4	32,768	512	33,280	524,288
Global average pool	256	-	-	0	-
FC1 (64 units) + ReLU6 + Dropout 0.2	64	16,448	-	16,448	16,384
FC2 (43 units)	43	2,795	-	2,795	2,752
Total				**66,827 (< 100,000)**	**4,162,240 (4.16 M)**

Why depthwise-separable convolution is cheaper: a standard k×k convolution costs $k^2 \cdot Cin \cdot Cout$ MACs per output position. The separable version costs $k^2 \cdot Cin + Cin \cdot Cout$, a ratio of $1/Cout + 1/k^2$. For k = 3 and Cout = 256 this is about 0.115, roughly 8.7× fewer MACs in that layer. For the whole network the saving is smaller (3.9×) because the standard stem convolution at 64×64 and the 1×1 pointwise layers still dominate.

Caveat for the discussion: Model B differs from Model A in two ways: depthwise-separable convolutions, and global average pooling instead of a flatten-FC head. Part of the 5.5× parameter reduction comes from the pooling change (it removes the 262k-parameter FC1), not only from the separable convolutions. Model B's convolutional part has 47,584 parameters against 97,680 in Model A's, a 2.1× reduction even though Model B is twice as wide at the last stage.

Memory-footprint estimate. fp32 weights: Model A ≈ 1,428 KB, Model B ≈ 261 KB. With int8 quantisation these become about 357 KB and 65 KB. The saved .pt files are slightly larger because they also store BN statistics. Measured saved-file sizes: Model A 1,437.4 KB, Model B 282.4 KB (slightly above 4 bytes per parameter because each file also stores the BatchNorm running statistics).

2.4 Activation functions: hardware-aware justification

- **ReLU (Model A) and ReLU6 (Model B)** are a single comparison (max(0, x), or min(max(0, x), 6)). They need no multiplier and no exponential, so they run in one cycle on a microcontroller or edge NPU, and need no look-up tables or floating-point unit.
- **Avoided on purpose:** sigmoid, tanh, softplus, Swish/SiLU and GELU all require exp() (or an approximation), which is expensive on devices without an FPU; hard-swish only partly avoids it.
- **ReLU6 for Model B** bounds activations to [0, 6]. A fixed, small dynamic range makes 8-bit fixed-point quantisation accurate (this is the reason MobileNets use it), so Model B can be deployed as int8. ReLU's unbounded range makes post-training quantisation harder, but Model A is the reference model and is not meant for deployment. This is a small difference between the models and is noted when comparing them.
- **Output layer:** softmax appears only inside the training loss. At inference the predicted class is the argmax of the logits, so no exponential is evaluated on the device.
- **BatchNorm** can be folded into the preceding convolution's weights and bias at deployment, so it adds no runtime cost.

3. Optimizer Selection and Tuning

Chosen optimizer: Adam, learning rate 1×10^{-3} ($\beta_1 = 0.9$, $\beta_2 = 0.999$, constant learning rate), batch size 128.

Why Adam: it keeps per-parameter adaptive step sizes, which matters here because Model B mixes very different layers (tiny depthwise kernels with 9 weights per channel, large pointwise and FC matrices). One global learning rate suits these poorly under plain SGD. Adam with 1×10^{-3} is the well-known robust default, converges in few epochs without per-model tuning, and works with BatchNorm. The extra optimizer state (two moments per weight) matters only during training, which runs on a workstation or GPU, not on the edge device, so it does not affect the deployed memory footprint.

Comparison protocol. Model B was trained for 20 epochs from the same initial weights (same seed), the same data order seed and the same augmentation, with: (a) Adam lr 1e-3, (b) plain SGD lr 0.01, (c) SGD with momentum 0.9, lr 0.01. A momentum sweep over {0, 0.5, 0.9, 0.99} at lr 0.01 isolates the momentum effect.

Result (Table 2 from the script):

Optimizer	Effective SGD step η/(1−β)	Best val acc	Final val acc	Min val loss	Epochs to 90% val acc	Test acc
Adam, lr 0.001	-	0.9876	0.9849	0.0466	7	0.9864
SGD, lr 0.01, β = 0	0.01	0.4275	0.4275	1.7583	not reached	0.4230
SGD, lr 0.01, β = 0.5	0.02	0.6596	0.6596	0.9951	not reached	0.6666
SGD, lr 0.01, β = 0.9	0.1	0.9786	0.9786	0.0714	10	0.9813
SGD, lr 0.01, β = 0.99	1.0	0.9907	0.9907	0.0335	7	0.9913

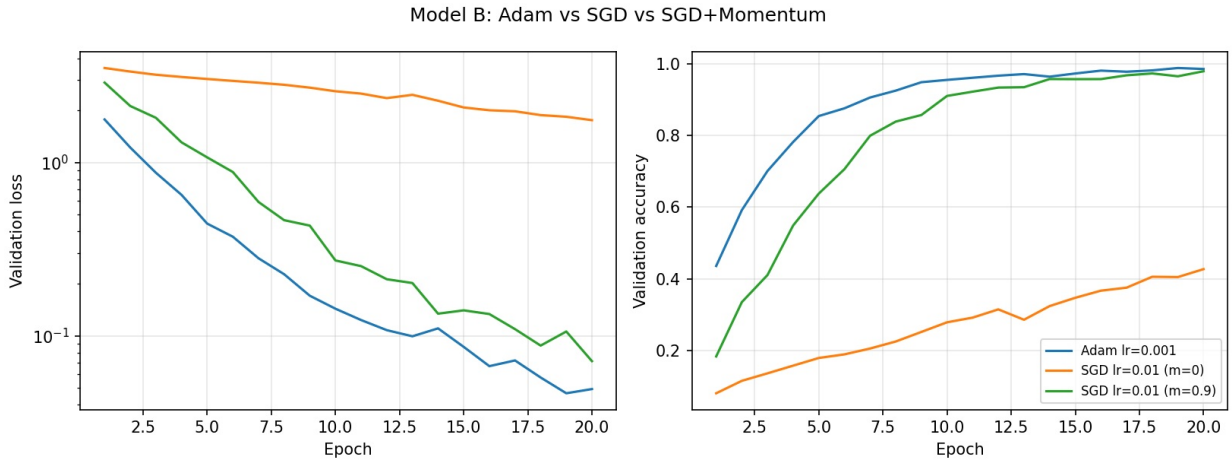


Figure 3: Model B, validation loss (log scale, left) and validation accuracy (right) for Adam (blue), plain SGD (orange) and SGD with momentum 0.9 (green).

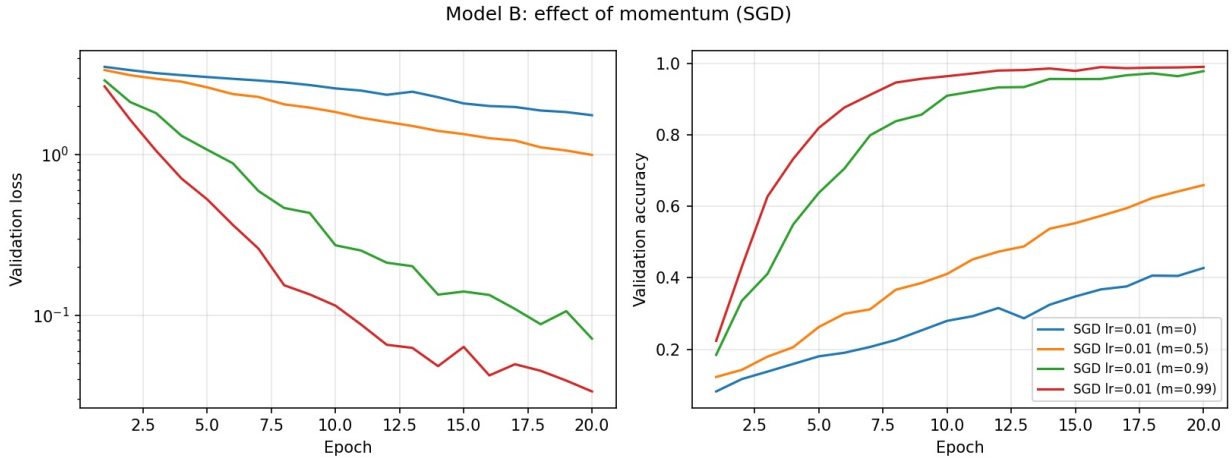


Figure 4: Effect of momentum for SGD with lr 0.01: β = 0 (blue), 0.5 (orange), 0.9 (green) and 0.99 (red). Adam is not drawn here; compare it using Table 2.

Impact of the momentum parameter. SGD with momentum keeps a velocity $v \leftarrow \beta \cdot v + g$ (g = gradient) and updates $w \leftarrow w - \eta \cdot v$. Two effects follow:

1. *Acceleration along consistent directions.* When successive gradients point the same way, v accumulates and the effective step approaches $\eta / (1 - \beta)$: 1× at β = 0, 2× at 0.5, 10× at 0.9, 100× at 0.99. Momentum therefore speeds up progress in shallow, consistent slopes such as the flat valleys of a loss surface.
2. *Damping of oscillations.* Components of the gradient that flip sign between steps (steep ravine walls, mini-batch noise) partially cancel in v, giving a smoother trajectory and a smoother loss curve.

The cost is overshoot. A large β with an unchanged η is effectively a much larger learning rate and can produce loss spikes or divergence (watch β = 0.99). Momentum also reacts slowly to a change of direction, so the loss can overshoot a minimum before settling.

Interpretation of our results: Plain SGD at lr 0.01 was far too slow: after 20 epochs validation accuracy was only 42.8%, with the loss still falling. Momentum helped at every step: β = 0.5 reached 66.0%, β = 0.9 reached 97.9% (90% at epoch 10) and β = 0.99 reached 99.1% (90% at epoch 7). Because the effective step is η/(1−β), these runs correspond to effective learning rates of 0.01, 0.02, 0.1 and 1.0. So the main effect of momentum here is that it makes up for a learning rate that was too small for plain SGD, and it also smooths the updates. The β = 0.99 run showed no instability (only small bumps in validation loss at epochs 15 and 17, similar in size to those of the other runs; see Figure 4), which BatchNorm helps to explain, since it keeps the network tolerant of large steps. Adam at its default lr 1×10^{-3} needed no tuning and was fastest at the start (43.6% validation accuracy after epoch 1, against 22.3% for β = 0.99). SGD with β = 0.99 caught up by about epoch 6–7 and finished higher: test accuracy 99.13% against 98.64%, validation loss 0.034 against 0.047.

Conclusion: We kept Adam for the main experiments because it was chosen in advance and works at its default learning rate with no tuning. The study shows, however, that SGD with momentum 0.99 was a slightly better optimizer for Model B in this setting (+0.5 points, about 29 test images, from a single seed, so partly noise is possible). We did not retrain Models A and B with it, to avoid selecting the optimizer on test-set results. Neither optimizer's learning rate was tuned, so this is a comparison of reasonable defaults, not of best-case performance.

4. Custom Model Training and Evaluation

Setup. Both models: 20 epochs, batch size 128, Adam (lr 1×10^{-3}), cross-entropy loss, the same data splits and augmentation, seed 42. After every epoch the validation loss and accuracy were recorded; the weights with the best validation accuracy were restored before testing, so the test set was used exactly once per model. Hardware: CPU only (no GPU), so all timings in this report are CPU timings. Training time per epoch is the mean wall-clock time of the training pass only.

4.1 Loss curves

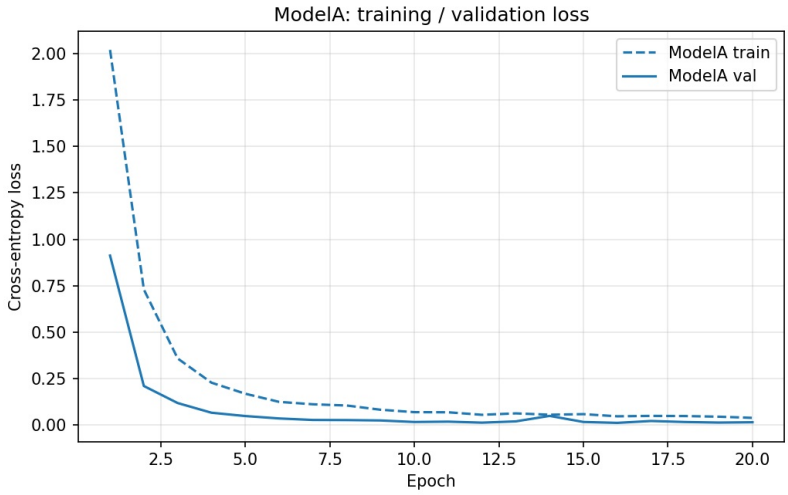


Figure 5: Model A, training (dashed) and validation (solid) cross-entropy loss per epoch.

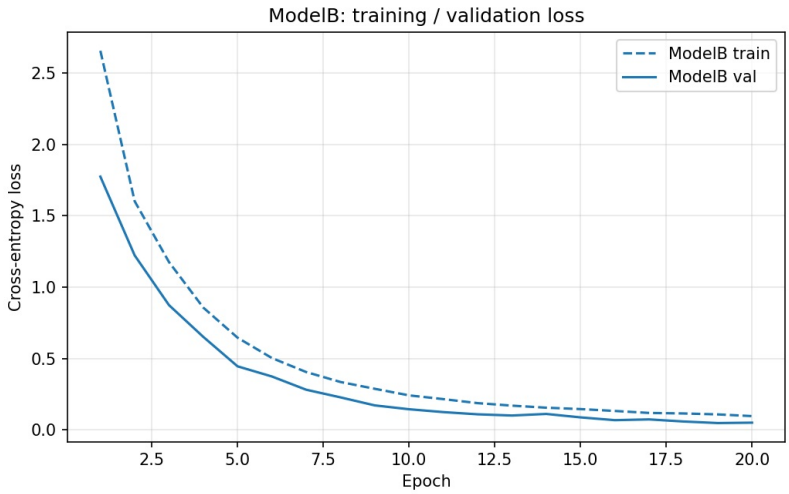


Figure 6: Model B, training (dashed) and validation (solid) cross-entropy loss per epoch.

Discussion: Neither model overfits. For Model A, validation loss falls from 0.91 to about 0.012–0.016 and validation accuracy stays above 99% from epoch 6. Validation loss sits below training loss throughout because augmentation and dropout are active only in training (final training accuracy 98.7%, validation 99.7%). Model A has essentially converged by epoch 10, apart from isolated validation dips (epoch 14: 98.2%). Model B learns more slowly: it needs 7 epochs to pass 90% validation accuracy (Model A needs 2), and its validation loss is still falling at epoch 20 (0.049; training accuracy 97.0%, validation 98.5%). So part of the final 1.2-point gap is probably that Model B is under-trained after 20 epochs, not only that it has less capacity. More epochs would likely narrow the gap, but we did not test this.

4.2 Test-set results

Definitions. Accuracy = correct / total. For class c: precision = TP / (TP + FP), the fraction of signs predicted as c that really are c; recall = TP / (TP + FN), the fraction of true c signs that were found. We report the **macro average** (unweighted mean over the 43 classes), which treats rare classes equally and is therefore stricter than accuracy on an imbalanced dataset. Per-class values are in per_class_ModelA.csv and

per_class_ModelB.csv.

Metric (test set)	Model A	Model B
Accuracy	0.9981	0.9864
Misclassified images (of 5,891)	11	80
Precision (macro)	0.9981	0.9861
Recall (macro)	0.9969	0.9848
F1 (macro)	0.9975	0.9853

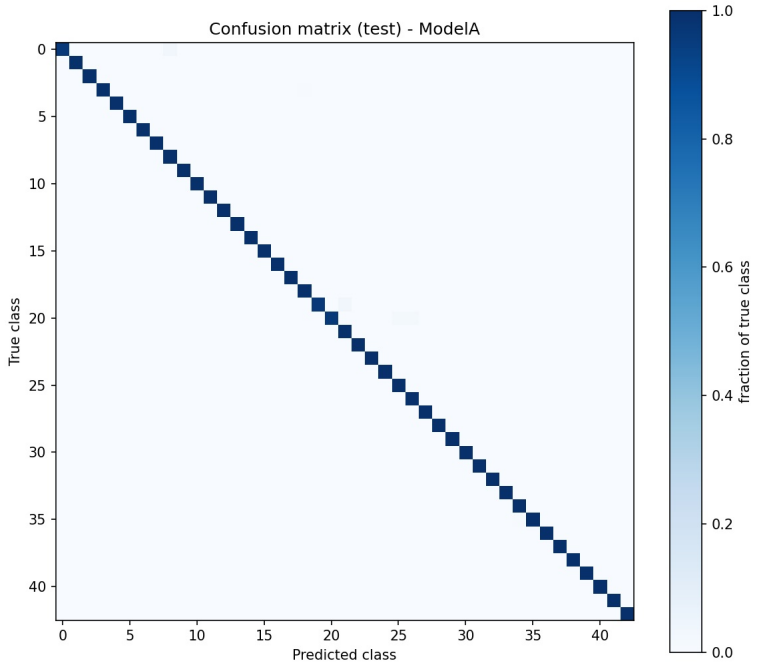


Figure 7: Model A, row-normalised confusion matrix on the test set. Almost all mass is on the diagonal.

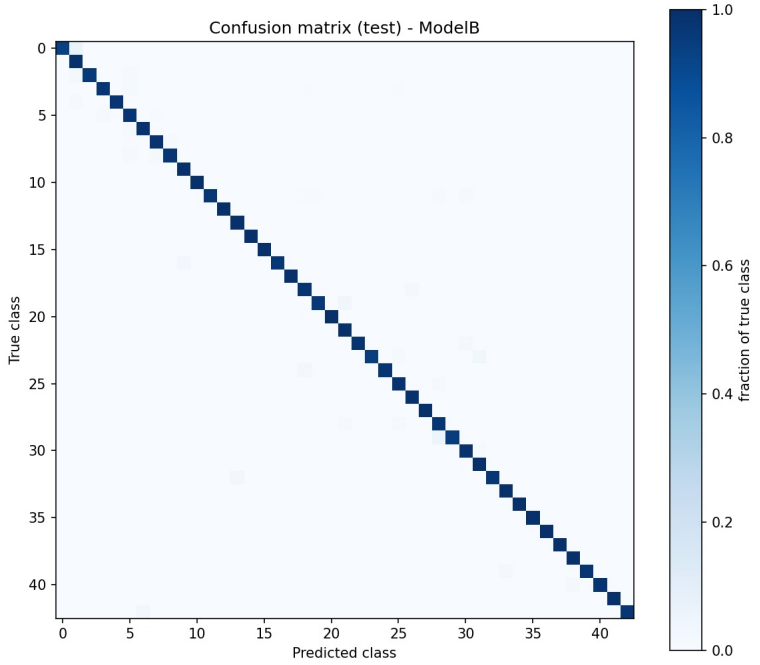


Figure 8: Model B, row-normalised confusion matrix on the test set. The faint off-diagonal cells are the 80 errors.

Confusion-matrix discussion: Model A made only 11 errors on 5,891 test images, and its most frequent confusion occurs just once (for example Ahead only → Turn left ahead, Speed limit 70 → Speed limit 100, Dangerous curve left → Double curve), so there is no systematic weakness. Model B made 80 errors. Its largest confusions are mostly between speed-limit signs: Speed limit 50 → 30 (6 times) and 50 → 80 (5), 70 → 30 (3), 80 → 60 (3), 60 → 80 (3), 120 → 100 (3). This supports the hypothesis that at 64×64 the digits on speed-limit signs differ

by only a few pixels, and the smaller model has less capacity to separate them. The other main confusions are General caution → Traffic signals (4) and Slippery road → Wild animals crossing (3). These are all warning triangles whose inner pictograms are small and similar at this resolution.

Per-class recall of Model B is lowest for Speed limit 20 (93.5%, 2 of 31 test images wrong), Slippery road (94.8%, 4 of 77), Bicycles crossing (95.0%, 2 of 40) and Speed limit 50 (96.5%, 12 of 338). The first three are rare classes, so one error moves their recall by 1.3 to 3.2 points and these values are statistically fragile. Speed limit 50 is the most reliable weak spot, because it has many test images and is the most frequent source of confusions.

4.3 Comparison table: Model A vs Model B

	Model A (standard)	Model B (depthwise-separable)
Total trainable parameters	365,499	66,827
MACs per image	16.19 M	4.16 M
Model size on disk, fp32 (KB)	1,437.4	282.4
Estimated size, int8 (KB)	356.9	65.3
Training time per epoch (s, CPU)	22.5	23.1
CPU inference latency, 1 thread (ms)	1.94	1.99
Test accuracy	99.81%	98.64%

4.4 Trade-offs when moving from standard to depthwise-separable convolutions

- **Parameters and memory (fixed by design).** Model B has 5.5× fewer parameters and a 5.1× smaller file (282.4 KB against 1,437.4 KB). Remember the caveat from Section 2.3: part of this comes from replacing the flatten-FC head with global average pooling.
- **Arithmetic cost.** MACs fall 3.9×. In the separable layers alone the saving approaches 1/Cout + 1/9 (about 8.7× for the 256-channel layer), but the standard stem convolution at full 64×64 resolution is now 43% of Model B's remaining compute.
- **MACs are not the same as speed.** In our measurements Model B was not faster: 23.1 s per epoch against 22.5 s for Model A, and 1.99 ms against 1.94 ms single-image CPU latency. The 3.9× MAC reduction therefore did not become a speed-up on this platform. Depthwise convolutions have low arithmetic intensity (few multiplications per byte moved), and PyTorch's CPU kernels for them are less optimised than for standard convolutions. During training, the augmentation and data handling are identical for both models and add a fixed cost. The saving should appear on hardware with efficient depthwise kernels or where energy per MAC dominates, but we did not measure that.
- **Accuracy.** Model A reached 99.81% and Model B 98.64% test accuracy, a gap of 1.17 percentage points (11 against 80 errors on 5,891 images; macro recall 99.69% against 98.48%). With a single seed we cannot give a confidence interval, but 69 extra errors is likely more than run-to-run variation, and Model B's loss was still falling at epoch 20 (Section 4.1). Because GTSRB signs have simple, high-contrast shapes, a small accuracy loss for a large resource saving is the expected trade.
- **Quantisation.** ReLU6 and BatchNorm folding make Model B a good int8 candidate. Depthwise layers are known to be more sensitive to quantisation error than standard layers, so the int8 accuracy should be checked on the real device. This was not measured here.
- **Training behaviour.** Model B converged more slowly (90% validation accuracy at epoch 7 against epoch 2 for Model A) but trained stably, while Model A showed occasional validation dips (for example epoch 14).

5. Comparison with Lightweight State-of-the-Art Networks

5.1 Chosen networks and why

- **MobileNetV2** (Sandler et al., 2018): inverted residual blocks with linear bottlenecks, built from the same depthwise-separable convolution as Model B. It is the standard mobile baseline and is supported by TFLite, ONNX and most NPU toolchains. This makes it the natural 'industry version' of Model B.
- **SqueezeNet 1.1** (Iandola et al., 2016): Fire modules (1×1 'squeeze' layer, then parallel 1×1 and 3×3 'expand' layers) with no depthwise convolutions and no fully connected layer. It is a deliberately different route to a small model, so the comparison covers two design philosophies.

5.2 Fine-tuning procedure

Both networks were initialised from ImageNet weights (torchvision IMAGENET1K_V1). The 1000-class output layer was replaced by a new 43-class layer (Linear $1280 \to 43$ for MobileNetV2, 1×1 convolution $512 \to 43$ for SqueezeNet). **All layers were fine-tuned** (not only the head) with Adam, lr 5×10^{-4}, for 20 epochs, on exactly the same train/validation/test split and augmentation as Models A and B. Inputs were normalised with the ImageNet mean and standard deviation, as the pretrained filters expect. Input size: 64×64, the same as for the custom models (no up-sampling).

Note the scale mismatch: both networks were pretrained on 224×224 images. At 64×64 the final MobileNetV2 feature map is 2×2, so fine-tuning has to adapt filters to far smaller objects.

5.3 Results

Parameter counts after replacing the head (confirmed by the script): MobileNetV2 = 3,504,872 − 1,281,000 + 55,083 = **2,278,955**; SqueezeNet 1.1 = 1,235,496 − 513,000 + 22,059 = **744,555**.

Metric (test set)	MobileNetV2	SqueezeNet 1.1
Total parameters	2,278,955	744,555
Model size, fp32 (MB)	8.92	2.86
MACs per image (M)	24.50	17.17
Training time per epoch (s, CPU)	108.9	31.5
CPU inference latency, 1 thread (ms)	6.86	3.54
Accuracy	0.9978	0.9946
Precision (macro)	0.9981	0.9947
Recall (macro)	0.9971	0.9941

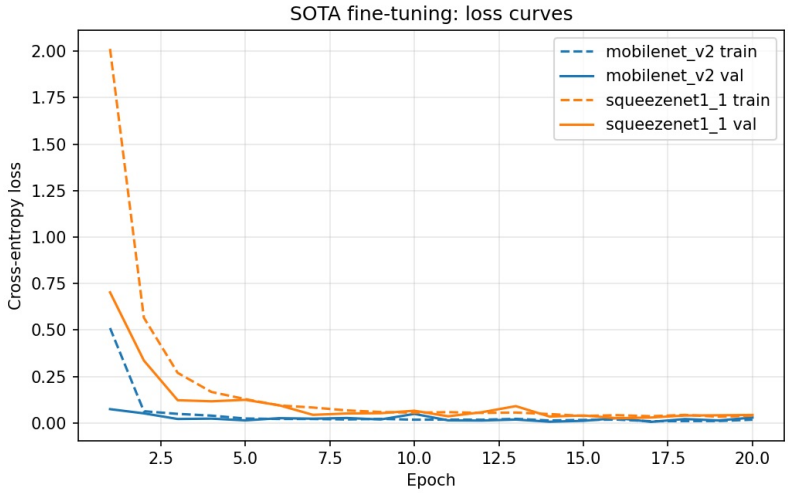


Figure 9: Fine-tuning loss curves (training dashed, validation solid). MobileNetV2 starts with a low validation loss in epoch 1. SqueezeNet starts higher and its validation loss is noisier in later epochs.

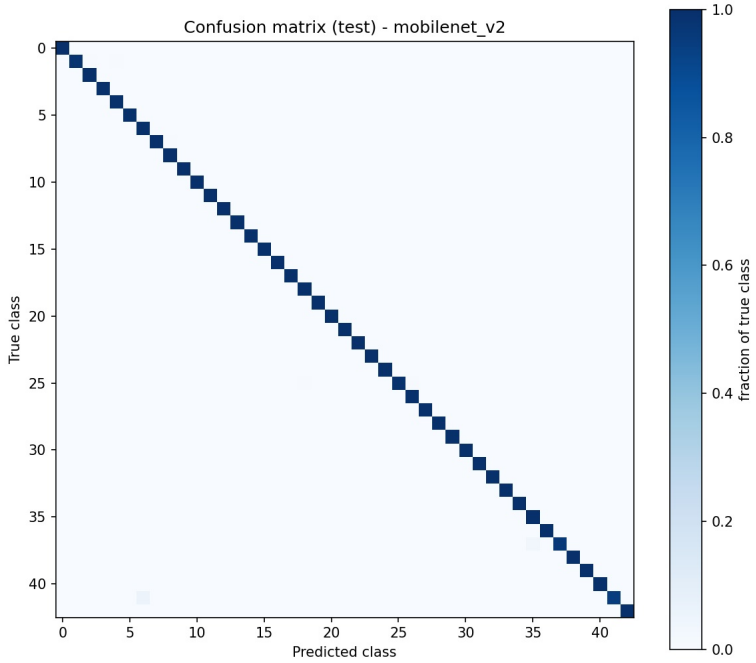


Figure 10: MobileNetV2, row-normalised confusion matrix on the test set.

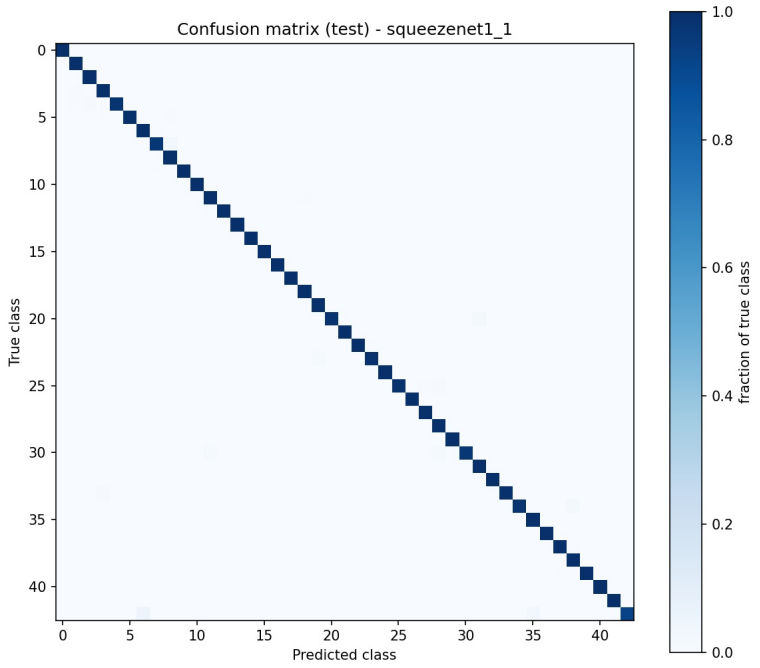


Figure 11: SqueezeNet 1.1, row-normalised confusion matrix on the test set.

Discussion: MobileNetV2 was the more accurate of the two (99.78% against 99.46% test accuracy; 13 against 32 errors on 5,891 images). It also converged fastest: 97.8% validation accuracy after the first epoch, thanks to the pre-trained features, while SqueezeNet reached 96.2% after epoch 3. The price is cost: MobileNetV2 has 3.1× the parameters of SqueezeNet, 1.4× the MACs, takes 3.5× longer per training epoch on our CPU (108.9 s against 31.5 s) and 1.9× longer per image (6.86 ms against 3.54 ms). SqueezeNet, built from Fire modules without depthwise convolutions, spends more MACs per parameter (17.2 M MACs for 0.74 M parameters). In later epochs both networks show noisy validation loss (for example MobileNetV2 between 0.006 and 0.049), which is why the best-validation checkpoint was used for testing. Both were more accurate than Model B (98.64%), and MobileNetV2 matched Model A (99.78% against 99.81%).

6. Final Comparison and Trade-off Discussion

	Model B (custom)	MobileNetV2	SqueezeNet 1.1	Model A (reference)
Trainable parameters	66,827	2,278,955	744,555	365,499
Model size, fp32 (MB)	0.28	8.92	2.86	1.40
Estimated size, int8 (MB)	0.06	2.17	0.71	0.35
MACs per image (M)	4.16	24.50	17.17	16.19
CPU latency, 1 thread (ms)	1.99	6.86	3.54	1.94
Test accuracy	98.64%	99.78%	99.46%	99.81%
Macro precision / recall	0.9861 / 0.9848	0.9981 / 0.9971	0.9947 / 0.9941	0.9981 / 0.9969

6.1 Accuracy, memory and compute

- **Accuracy.** Fine-tuned MobileNetV2 reached 99.78% test accuracy, SqueezeNet 1.1 99.46% and our Model B 98.64% (Model A, for reference, 99.81%). Model B gives up 1.14 points to MobileNetV2 and 0.82 points to SqueezeNet, in exchange for 34× and 11× fewer parameters. In errors on 5,891 test images: 80 for Model B, 32 for SqueezeNet, 13 for MobileNetV2. Transfer learning clearly helped, even though ImageNet photographs look very different from traffic signs: MobileNetV2 reached 97.8% validation accuracy after one epoch. But the from-scratch Model A (365k parameters) matched MobileNetV2 (2.28 M), so on this dataset capacity beyond a few hundred thousand parameters bought almost nothing.
- **Memory.** Model B's fp32 file is 0.28 MB, against 8.92 MB for MobileNetV2 (32×) and 2.86 MB for SqueezeNet (10×). This decides feasibility, not just cost. A typical microcontroller (e.g. a Cortex-M4/M7 class part) has on the order of 256–512 KB of SRAM and 1–2 MB of flash. Model B's int8 weights (about 65 KB) fit comfortably. SqueezeNet's (about 0.7 MB) would take most of the flash. MobileNetV2's (about 2.2 MB) do not fit without pruning, a smaller width multiplier or external memory.
- **Activation memory.** Weights are only half of the memory story. At run time the largest feature map must also fit in RAM. In Model B the stem output is 16×64×64 = 65,536 values (64 KB in int8), plus the 12 KB input, so about 76 KB peak before layer-by-layer buffer reuse. A stride-2 stem would cut this by 4× and also remove most of the stem's 1.77 M MACs. This is a concrete next design step.
- **Compute.** MobileNetV2 needs 24.50 M MACs per image, 5.9× Model B's 4.16 M. SqueezeNet needs 17.17 M (4.1×). Measured single-thread CPU latency was 1.99 ms for Model B, 3.54 ms for SqueezeNet and 6.86 ms for MobileNetV2, i.e. 1.8× and 3.4× slower. The latency gap is smaller than the MAC gap because, as in Section 4.4, PyTorch does not turn the MAC saving of depthwise

layers into speed on a CPU. On a microcontroller, where energy per MAC dominates, the MAC ratio is likely the better guide, but we did not measure that. Training cost per epoch on our CPU: 23.1 s (Model B), 31.5 s (SqueezeNet), 108.9 s (MobileNetV2).

6.2 Custom sub-100k model versus pre-trained lightweight model

Advantages of the custom model

- Meets a hard budget by construction (66,827 parameters, ≈65 KB int8) rather than by compressing a network afterwards.
- Every layer is chosen for the device: ReLU6 and BN-folding, no exponentials, small peak activation size, easy to convert to a C array for a microcontroller.
- Can be matched to the task. GTSRB has 43 classes of simple shapes, so a network with 2.3 M parameters is more capacity than the problem needs.
- No dependence on ImageNet weights or their licence, and no scale mismatch with a 64×64 input.

Limitations of the custom model

- Training from scratch needs more data and more epochs, and gives no benefit from transfer learning. On a harder or smaller dataset the gap to pre-trained models would widen.
- Architecture and hyper-parameters had to be designed and tuned by us. The pre-trained models encode years of architecture search.
- Less robust to distribution shift (new lighting, camera, weather) than features learned from 1.2 M ImageNet images. We did not test robustness to distribution shift, so this remains a hypothesis.
- Narrow scope: a different task means redesigning and retraining.

Advantages of the pre-trained lightweight models

- High accuracy with little tuning; fast convergence from good initial weights.
- Mature deployment tooling (TFLite, ONNX, vendor NPU compilers, quantisation recipes).

Limitations of the pre-trained models

- 11–34× more parameters than Model B (see table), so they do not fit the smallest devices, and carry capacity the task does not use.
- Fixed design: scaling them down further means pruning or width multipliers, which changes the pre-trained weights.

6.3 Conclusion and recommendation

Conclusion: For a microcontroller-class target we would deploy the custom Model B. It gives 98.64% test accuracy with 66,827 parameters (about 65 KB in int8), which is 34× fewer parameters and 5.9× fewer MACs than MobileNetV2, at a cost of 1.14 accuracy points. If the device has megabytes of memory and an accelerator (for example a Raspberry Pi) and accuracy matters most, fine-tuned MobileNetV2 is the safer choice, and it needs no architecture design. The result that stands out is Model A: a from-scratch standard CNN with 365,499 parameters matched MobileNetV2's accuracy (99.81% against 99.78%). A better custom design therefore probably lies between A and B, for example Model B made wider, or Model A with a global-average-pooling head. We did not test this.

Scenario	Better choice
Microcontroller, under 512 KB RAM, under 1 MB flash	Custom Model B (int8)
Raspberry Pi / phone-class device, accuracy is the priority	Fine-tuned MobileNetV2
Little training data, a new task, short development time	Fine-tuned pre-trained model
Strict energy budget per inference	Custom Model B, or a further reduced version of it

7. Limitations and Threats to Validity

- **Possible train/test leakage.** GTSRB training images come in sequences of about 30 frames of the same physical sign. Our random stratified split can place near-identical frames of one sign in both training and test sets, which inflates accuracy relative to the official benchmark (whose test set contains different physical signs). Absolute accuracies should be read with this in mind. The comparison between models remains fair because all models use the same split. The very high accuracies (up to 99.8%) may partly reflect this.
- **Single run per model** (seed 42). Differences of a few tenths of a percent are within run-to-run noise.
- **Timing.** All timings were measured on a CPU (no GPU); training time per epoch also includes the augmentation and data pipeline. CPU latency is measured on a general-purpose CPU and is only a proxy for a microcontroller.
- **int8 sizes are estimates** (one byte per parameter), not measured after real quantisation.
- **Low-resolution input** (64×64) disadvantages the ImageNet-pretrained networks more than the custom ones.

8. Contribution of Group Members

Member	Responsibilities	Report sections
ABISHEK L. (230016K)	Architecture design of Models A and B; parameter and MAC calculations; activation-function justification; limitations	2, 7
LUCKSHAN S.M. (230380T)	Data preparation (download, resizing, stratified split, augmentation, dataset sheet); optimizer study (Adam, SGD, momentum sweep); shared repository setup; final report assembly, PDF export and submission	1, 3
SAMPAVI G. (230572J)	Training and evaluation of Models A and B; loss curves; confusion matrices, precision and recall; Model A versus Model B comparison	4
SANTHOSH S. (230581K)	Fine-tuning and evaluation of MobileNetV2 and SqueezeNet 1.1; final comparison and trade-off discussion	5, 6

All members reviewed the whole report and the code. Each member committed their own part to the shared repository from their own GitHub account.

References

[1] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

[2] K. Fukushima, ‘Cognitron: A self-organizing multilayered neural network,’ *Biological Cybernetics*, vol. 20, pp. 121–136, 1975.

[3] D. H. Hubel and T. N. Wiesel, ‘Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex,’ *J. Physiology*, vol. 160, p. 106, 1962.

[4] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, ‘Gradient-based learning applied to document recognition,’ *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[5] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, ‘Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition,’ *Neural Networks*, vol. 32, pp. 323–332, 2012.

[6] A. G. Howard et al., ‘MobileNets: Efficient convolutional neural networks for mobile vision applications,’ arXiv:1704.04861, 2017.

[7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, ‘MobileNetV2: Inverted residuals and linear bottlenecks,’ in *CVPR*, 2018.

[8] F. N. Iandola et al., ‘SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size,’ arXiv:1602.07360, 2016.

[9] D. P. Kingma and J. Ba, ‘Adam: A method for stochastic optimization,’ in *ICLR*, 2015.

[10] S. Ioffe and C. Szegedy, ‘Batch normalization: Accelerating deep network training by reducing internal covariate shift,’ in *ICML*, 2015.

[11] I. Sutskever, J. Martens, G. Dahl and G. Hinton, ‘On the importance of initialization and momentum in deep learning,’ in *ICML*, 2013.

[12] F. Chollet, ‘Xception: Deep learning with depthwise separable convolutions,’ in *CVPR*, 2017.

Appendix: How to Reproduce

1. `pip install torch torchvision scikit-learn matplotlib pillow numpy` (requirements.txt is provided).
2. `python gtsrb_edge_cnn.py` (a GPU is recommended; the full run took roughly 1.5 hours on our CPU-only machine). Quick smoke test without downloads: `python gtsrb_edge_cnn.py --epochs 2 --fake-data --skip-sota`.
3. All figures, confusion matrices, per-class CSV files, results.json and results_tables.md are written to ./outputs.